3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

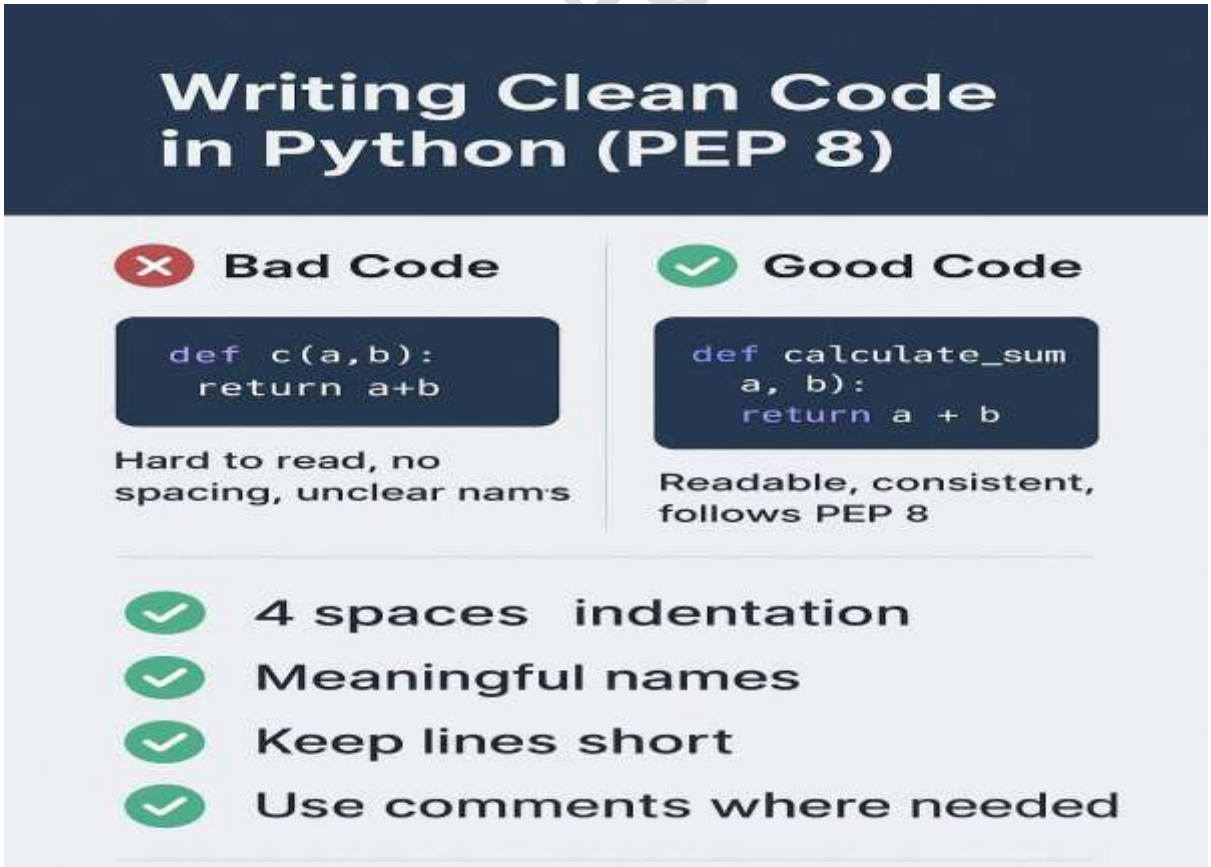
Daily Teaching : 2 Hours

Total Classes : 100

FOUNDATION

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 2 – Python Programming Best Practices



Learning Objectives

By the end of this class, students will be able to:

- Understand what **clean code** means.
- Write Python programs that are easy to read and understand.
- Use meaningful variable and function names.
- Follow proper indentation and spacing.
- Write useful comments and documentation.
- Avoid unnecessary or repeated code.
- Break large programs into functions.
- Identify and improve poorly written Python code.
- Apply professional Python programming habits.

3DWEBSOFT FOUNDATION

1. What Are Programming Best Practices?

Programming best practices are recommended techniques that help programmers write code that is:

- Easy to read
- Easy to understand
- Easy to test
- Easy to debug
- Easy to modify
- Reusable
- Reliable
- Maintainable

A program can work correctly but still be **poorly written**.

Example

Both programs can produce the same output:

Poor style:

```
x=10
y=20
z=x+y
print(z)
```

Better style:

```
first_number = 10
second_number = 20

total = first_number + second_number

print(total)
```

The second program is easier for another programmer to understand.

2. What is Clean Code?

Clean code is code written in a simple, organized and understandable way.

Think about your classroom.

If books, bags and notebooks are scattered everywhere, finding something takes time.

If everything is organized properly, you can quickly find what you need.

Code works in a similar way.

3DWEBSOFT FOUNDATION



Writing Clean Code in Python (PEP 8)

Bad Code

```
def c(a,b):
  return a+b
```

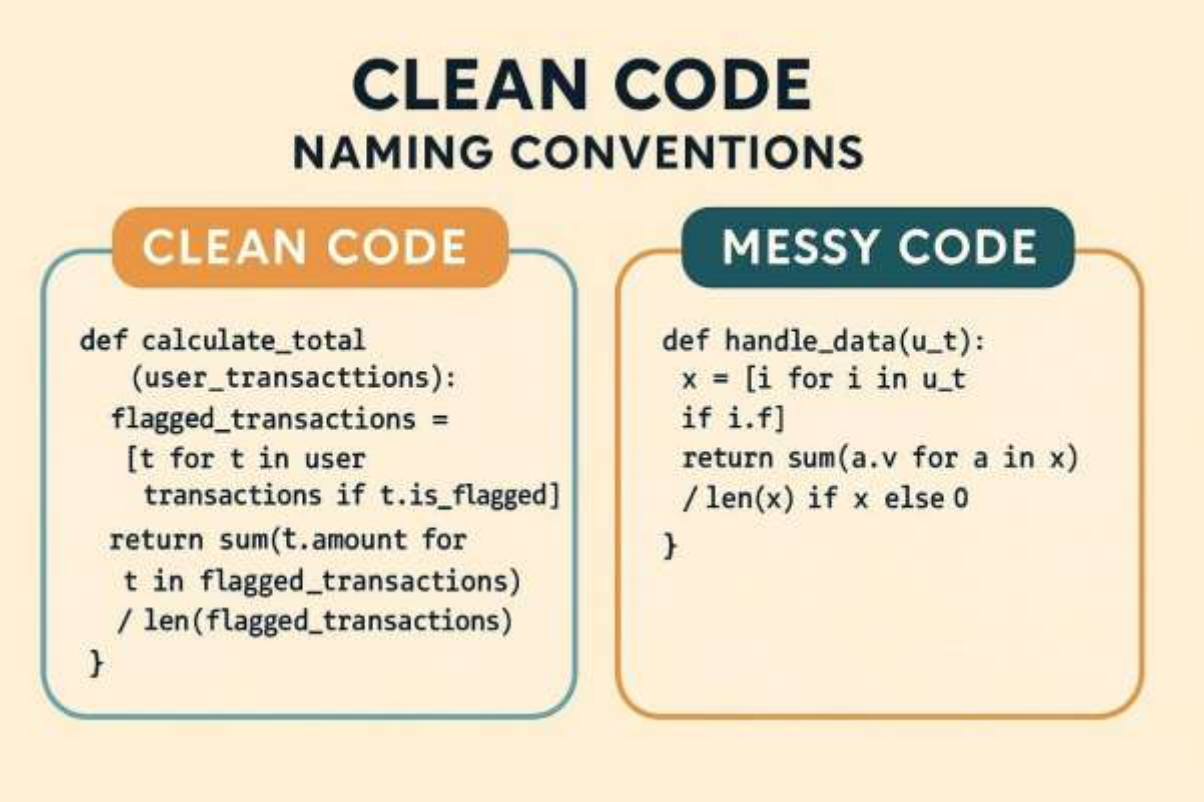
Hard to read, no spacing, unclear nams

Good Code

```
def calculate_sum
   a, b):
   return a + b
```

Readable, consistent, follows PEP 8

- 4 spaces indentation
- Meaningful names
- Keep lines short
- Use comments where needed



Clean Code

student_name = "Aisha"

student_marks = 92

print(student_name)

print(student_marks)

Messy Code

a="Aisha";b=92;print(a);print(b)

Both may work, but the first is much easier to read.

3. Use Meaningful Variable Names

Variable names should describe the information they contain.

Poor Naming

3DWEBSOFT FOU

```
x = 18
y = "Rahul"
z = 95
```

What do x, y and z mean?

Another programmer has to guess.

Better Naming

```
age = 18
student_name = "Rahul"
marks = 95
```

Now the purpose is immediately clear.

More Examples

Poor Name	Better Name
x	student_age
a	total_marks
n	student_name
p	product_price
r	exam_result

4. Follow Python Naming Conventions

Python commonly uses **snake_case** for variables and functions.

Correct

```
student_name = "Ravi"
total_marks = 450
calculate_average()
```

3DWEBSOFT FOUNDATION

Avoid

studentName = "Ravi"

totalMarks = 450

CalculateAverage()

For this course, students should consistently use:

lowercase_words_separated_by_underscores

Example:

student_age

total_price

calculate_marks

display_result

5. Proper Indentation

Python uses indentation to identify blocks of code.

Correct

```
marks = 85


if marks >= 40:
    print("Pass")
```

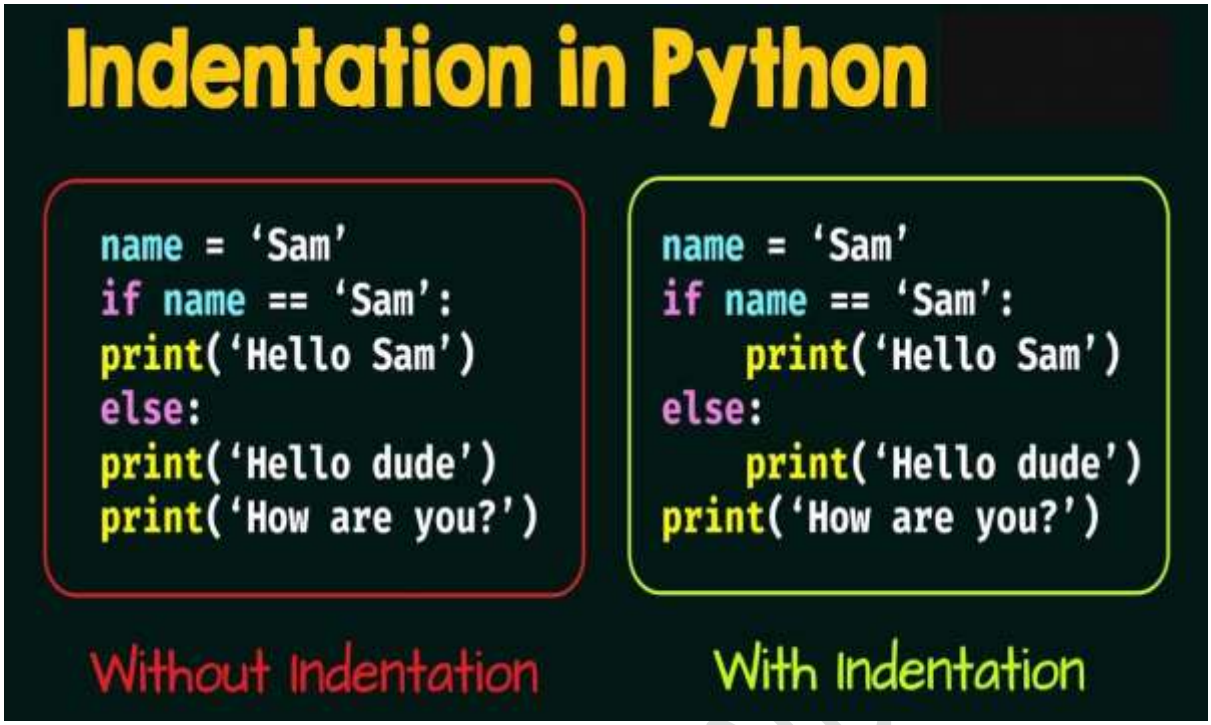
Incorrect

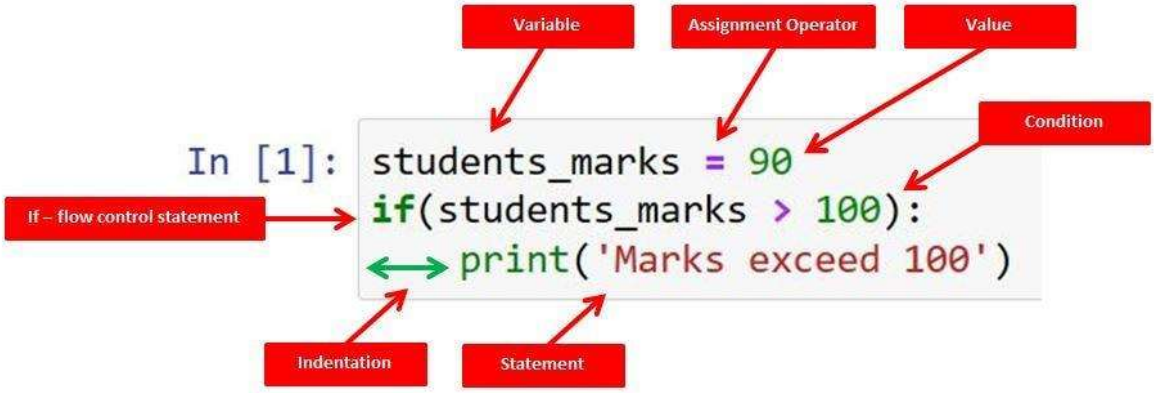
```
marks = 85

if marks >= 40:
print("Pass")
```

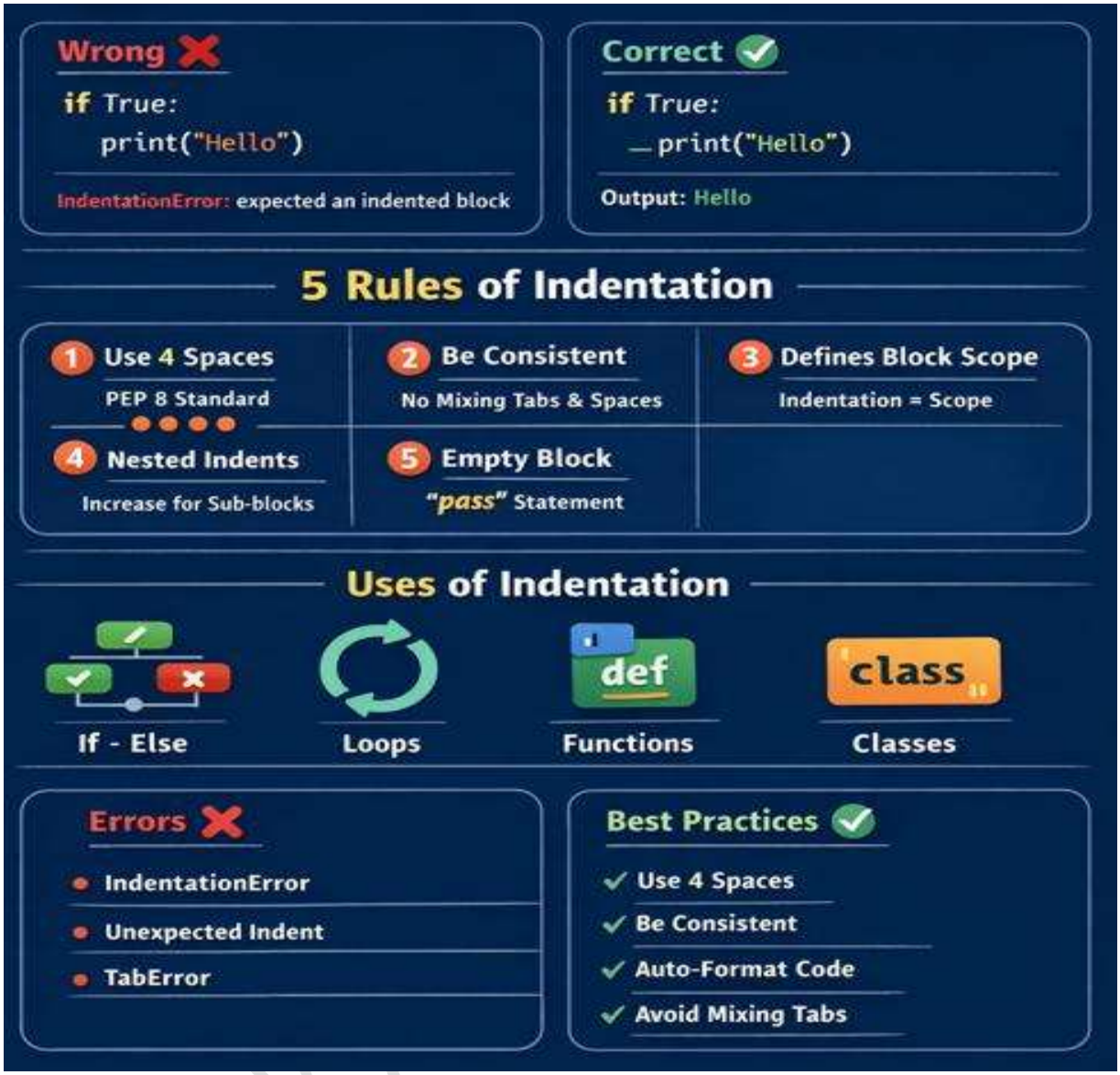
The second program produces an indentation error.

3DWEBSOFT FOUNDATION


Indentation in Python
name = 'Sam'
if name == 'Sam':
print('Hello Sam')
else:
print('Hello dude')
print('How are you?')
name = 'Sam'
if name == 'Sam':
print('Hello Sam')
else:
print('Hello dude')
print('How are you?')
Without Indentation
With Indentation


Variable
Assignment Operator
Value
Condition
In [1]:
students_marks = 90
if(students_marks > 100):
print('Marks exceed 100')
If – flow control statement
Indentation
Statement

3DWEB



Best Practice

Use **4 spaces** for each indentation level.

Example:

```
if marks >= 40:
    if marks >= 75:
        print("Very Good")
```

6. Keep Code Properly Spaced

Spacing improves readability.

Difficult to Read

```
total=marks1+marks2+marks3
average=total/3
```

Easier to Read

```
total = marks1 + marks2 + marks3
average = total / 3
```

Spaces around operators make programs easier to understand.

7. Use Comments

Comments explain code to human readers.

Python uses # for a single-line comment.

```
# Calculate the total marks
total_marks = maths + science + python
```

Another Example

```
# Check whether the student passed
if marks >= 40:
    print("Pass")
```

Comments should explain **why** something is being done when the reason isn't obvious—not repeat every line of code.

Unnecessary Comment

```
# Store 10 in age
age = 10
```

Useful Comment

```
# Minimum age required to participate in the competition
```

3DWEBSOFT FOUNDATION

```
minimum_age = 15
```

8. Use Functions to Organize Code

Large programs should be divided into smaller functions.

Without Functions

```
name = input("Enter name: ")
marks1 = int(input("Enter Maths marks: "))
marks2 = int(input("Enter Science marks: "))

total = marks1 + marks2
average = total / 2

print(name)
print(total)
print(average)
```

This is fine for a small program, but larger applications become difficult to manage.

Using Functions

```
def calculate_total(mark1, mark2):
    return mark1 + mark2


def calculate_average(total):
    return total / 2
```

3DWEBSOFT FOUNDATION

```
maths = int(input("Enter Maths marks: "))
science = int(input("Enter Science marks: "))

total = calculate_total(maths, science)
average = calculate_average(total)

print("Total:", total)
print("Average:", average)
```

Benefits

Functions provide:

- Reusability
- Organization
- Easier testing
- Easier debugging
- Better readability

9. Avoid Repeating Code – DRY

A popular programming principle is:

DRY = Don't Repeat Yourself

Suppose you write:

```
print("Welcome to Python")
print("Welcome to Python")
print("Welcome to Python")
```

For a larger program, repeated logic should be placed into a function.

3DWEBSOFT FOUNDATION

```
def welcome():
    print("Welcome to Python")



welcome()
welcome()
welcome()
```

Now the logic exists in one place.

10. Use Constants for Fixed Values

If a value should remain fixed, give it a meaningful name.

Example

```
PASS_MARKS = 40


marks = 65


if marks >= PASS_MARKS:
    print("Pass")
```

Instead of repeatedly writing:

```
if marks >= 40:
```

Using a named constant makes the purpose clearer.

11. Validate User Input

Programs should not blindly trust user input.

Example

3DWEBSOFT FOUNDATION

```
age = int(input("Enter your age: "))


if age < 0:
    print("Invalid age")
else:
    print("Age:", age)
```

A better program anticipates incorrect input.

Later in the course, students will learn **exception handling** for more robust input processing.

12. Avoid Extremely Long Functions

A function should generally have a clear responsibility.

Difficult Structure

Imagine one function containing:

- User input
- Calculations
- File handling
- Database operations
- Report generation
- Printing

This becomes difficult to understand.

Better Structure

```
def get_student_details():
    pass
```

3DWEBSOFT FOUNDATION

```
def calculate_result():
    pass


def save_result():
    pass


def display_result():
    pass
```

Each function has a specific job.

13. Use Modules

As programs become larger, divide code into modules.

Example:

```
student_project/
|
 ├── main.py
 ├── students.py
 ├── calculations.py
 └── reports.py
```

main.py can use functions from other files.

```
from calculations import calculate_average
```

This keeps projects organized.

3DWEBSOFT FOUNDATION

14. Use Simple Code

When two solutions work, prefer the one that is easier to understand.

Example

Instead of creating unnecessarily complicated logic:

```
marks = 85


if marks >= 40:
    result = True
else:
    result = False
```

You can write:

```
marks = 85
result = marks >= 40
```

Simple code is often easier to maintain.

15. Practical Activity 1 – Improve the Code

Give students this code:

```
n="Rahul"
a=17
m1=85
m2=90
m3=80
t=m1+m2+m3
av=t/3
```

3DWEBSOFT FOUNDATION

```
print(n)

print(a)

print(t)

print(av)
```

Student Task

Improve it using:

- Meaningful names
- Proper spacing
- Comments
- Better formatting

Suggested Answer

```
# Student information

student_name = "Rahul"

student_age = 17


# Student marks

maths_marks = 85

science_marks = 90

python_marks = 80


# Calculate result

total_marks = maths_marks + science_marks + python_marks

average_marks = total_marks / 3


print("Student:", student_name)
```

3DWEBSOFT FOUNDATION

```
print("Age:", student_age)
print("Total Marks:", total_marks)
print("Average:", average_marks)
```

16. Practical Activity 2 – Create a Clean Calculator

Ask students to create a calculator using functions.

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b


first_number = float(input("Enter first number: "))
second_number = float(input("Enter second number: "))
```

3DWEBSOFT FOUNDATION

```
print("Addition:", add(first_number, second_number))
print("Subtraction:", subtract(first_number, second_number))
print("Multiplication:", multiply(first_number, second_number))
print("Division:", divide(first_number, second_number))
```

Student Challenge

Add:

- Modulus
- Power
- Floor division

17. Practical Activity 3 – Student Grade Program

Students should create a clean program that calculates a grade.

```
PASS_MARKS = 40


marks = float(input("Enter your marks: "))


if marks >= 90:
    grade = "A+"
elif marks >= 75:
    grade = "A"
elif marks >= 60:
    grade = "B"
elif marks >= PASS_MARKS:
    grade = "C"
else:
```

3DWEBSOFT FOUNDATION

```
    grade = "F"


print("Grade:", grade)
```

Discussion

Ask students:

Why did we create PASS_MARKS instead of repeatedly writing 40?

Expected answer:

It makes the program easier to understand and easier to change later.

18. Interactive Activity – Code Detective

Display this code:

```
def calc(a,b):
 x=a+b
 return x


n1=10
n2=20
result=calc(n1,n2)
print("Result:",result)
```

Students identify:

1. Naming could be improved.
2. Spacing could be improved.
3. Indentation should follow the standard 4-space convention.
4. The function name could be more descriptive.

Improved Version

3DWEBSOFT FOUNDATION

```
def calculate_total(first_number, second_number):
    total = first_number + second_number
    return total



first_number = 10
second_number = 20


result = calculate_total(first_number, second_number)


print("Result:", result)
```

19. Classroom Challenge – Clean Code Competition

Divide students into small teams.

Give each team a poorly written program.

Team Task

Students must improve the program using:

Meaningful names
Correct indentation
Proper spacing
Functions
Comments where useful
Avoid repeated code
Simple logic

Scoring

3DWEBSOFT FOUNDATION

Skill	Marks
Meaningful names	10
Indentation	10
Formatting	10
Functions	10
Code organization	10
Total	**50**

20. Best Practices Checklist

Before submitting a Python program, students should ask:

Clean Code Checklist

- Are my variable names meaningful?
- Is my code properly indented?
- Did I use appropriate spacing?
- Did I avoid unnecessary repetition?
- Can I divide my program into functions?
- Are my comments useful?
- Is the code easy to understand?
- Did I handle possible errors?
- Did I test the program?
- Can another student understand my code?

3DWEBSOFT FOUNDATION

Class Quiz

Q1. What does DRY mean?

Answer: Don't Repeat Yourself.

Q2. Which naming style is commonly used for Python variables?

A. PascalCase
B. snake_case
C. UPPERCASEONLY
D. hyphen-case

Answer: B

Q3. How many spaces are conventionally used for one indentation level?

Answer: 4 spaces.

Q4. What symbol is used for a single-line comment?

Answer: #

Q5. Why are functions useful?

Answer: They organize code and allow code to be reused.

Q6. Which is better?

x = 90

or

student_marks = 90

Answer: student_marks = 90, because the name clearly explains the value.

Homework

Basic

Create a clean Python program that accepts:

- Student name
- Age
- Class
- Three subject marks

Then display:

3DWEBSOFT FOUNDATION

- Total
- Average
- Grade
- Pass/Fail

Advanced

Create at least **three functions**:

get_student_details()

calculate_result()

display_result()

Students should apply all the best practices learned in today's class.

2-Hour Class Plan

Time	Activity
0–10 min	Previous class revision
10–30 min	What is clean code?
30–45 min	Naming, indentation & formatting
45–60 min	Comments, functions & DRY
60–80 min	Teacher demonstration
80–105 min	Practical coding
105–115 min	Code Detective activity
115–120 min	Quiz & homework

Key Message for Students

3DWEBSOFT FOUNDATION

Good programmers don't just write code that works. They write code that other people can understand, maintain and improve.

3DWEBSOFT FOUNDATION